

Relatório lista 4 estruturas de dados

1. Representação da Trie

A estrutura da Trie foi implementada usando a classe `TrieNode`. Cada nó possui um vetor de ponteiros chamado `children`, um booleano `isEndOfTitle` e um ponteiro `Game* game` que guarda o jogo associado. O alfabeto escolhido foi de tamanho 36, correspondente a 26 letras e os 10 algarismos de 0 a 9. Desse modo, cada `trieNode` tem um array de 36 filhos. O mapeamento dos caracteres foi feito por uma função auxiliar `charToIndex` que associa com comparações cada letra a sua respectiva contagem no alfabeto, começando em 0, e cada algarismo de 26 a 35. Caracteres não encaixados nos padrões receberam índice -1. Aliado a esse método, implementamos também a função `toSearchKey`, que normaliza os títulos dos jogos para os padrões da Trie, ou seja, sem espaço e minúsculo.

2. Funcionamento dos métodos

Os métodos `toSearchKey` e `insert` trabalham juntos para inserir jogos na Trie. O `toSearchKey` normaliza o título, removendo espaços e convertendo letras maiúsculas para minúsculas. Em seguida, o `insert` usa essa chave para percorrer a Trie caractere por caractere, criando novos nós quando necessário. Ao final do percurso, o último nó é marcado como fim de título e armazena o ponteiro para o jogo correspondente.

O método `contains` verifica se um título exato existe na Trie. Assim como na inserção, o título recebido é convertido para chave de busca. Em seguida, a Trie é percorrida caractere por caractere. Se em algum momento o caminho necessário não existir, o método retorna `false`. Caso o caminho exista até o final, o método verifica se o último nó está marcado como final de título. Se estiver, retorna `true`; caso contrário, retorna `false`.

O método `sortResults` é responsável por ordenar os jogos encontrados pelo autocomplete de acordo com os critérios definidos no enunciado: primeiro, maior popularidade; em caso de empate, ordem alfabética pela chave de busca do título. A ordenação foi implementada manualmente utilizando o algoritmo **Bubble Sort**. Esse algoritmo percorre o vetor várias vezes, comparando pares de elementos vizinhos. Sempre que dois jogos estão fora da ordem desejada, eles são trocados de posição. No código, essa comparação é feita com o auxílio do método `comesBefore`. Um jogo deve vir antes de outro se tiver popularidade maior. Caso os dois jogos tenham a mesma popularidade, o critério de desempate é a comparação entre as chaves de busca dos títulos, geradas pelo método `toSearchKey`. Dessa forma, o Bubble Sort

garante que, ao final do processo, o vetor esteja ordenado conforme as regras do autocomplete.

O método `autocomplete` recebe um prefixo e um número `k`. Caso $k \leq 0$, o método retorna um vetor vazio. Primeiro, o prefixo é convertido para chave de busca. Depois, a Trie é percorrida até o nó correspondente ao final desse prefixo. Se o caminho do prefixo não existir, significa que nenhum jogo começa com aquele prefixo, então o método retorna um vetor vazio. Caso o prefixo seja encontrado, o método chama uma função auxiliar para coletar todos os jogos armazenados na subárvore abaixo daquele nó. Isso garante que o autocomplete use a Trie como estrutura principal de busca, sem percorrer o array original de jogos. Após coletar os jogos, o vetor de resultados é ordenado manualmente pelo método `sortResults`.

3. Análise de custo

Na análise de custos, a operação `insert` possui custo $O(L)$, pois chama nela a função `toSearchKey`, que tem custo proporcional ao tamanho do título, e depois faz um for percorrendo os L caracteres da chave gerada. Logo, temos algo como $O(L) + O(L) = O(L)$. Por motivos semelhantes, a função `contains` também possui custo $O(L)$: novamente se chama `toSearchKey` e depois percorre-se no pior dos casos os L caracteres da chave gerada, gerando $O(L)$.

O `autocomplete`, por outro lado, é mais delicado. Começa-se convertendo o prefixo para formato de chave, gerando $O(p)$. Depois percorre-se o prefixo na árvore, gerando novamente $O(p)$. Após isso, ele chama em si a função auxiliar `collectGames` que faz um DFS a partir do nó correspondente ao prefixo na subárvore abaixo coletando todos os jogos que encontra. Essa parte possui custo $O(s)$. Por fim, chama-se a função `sortResults`, que é um **Bubble Sort** sobre jogos encontrados que, considerando que compara usando a função auxiliar `comesBefore` de custo $O(L)$, tem ordem $O(Lr^2)$. Assim, o custo total dessa função é $O(p+s+Lr^2)$.

4. Comparação e uso de memória

Uma solução ingênua para o autocomplete seria percorrer todo o array de jogos a cada busca, verificando quais títulos começam com o prefixo informado. Se houver n jogos e cada título tiver tamanho médio L , essa abordagem teria custo aproximado de $O(n \cdot L)$, além do custo adicional de ordenar os resultados encontrados. A solução com Trie evita essa busca completa, pois percorre apenas os caracteres do prefixo e depois explora a subárvore correspondente, o que é mais eficiente quando a base de jogos é grande ou quando o prefixo reduz bastante o número de resultados.

Por outro lado, a Trie utiliza mais memória do que um simples array, já que cada nó armazena um vetor de ponteiros para seus possíveis filhos. Neste projeto, cada nó possui 36 ponteiros, correspondentes às letras de **a** a **z** e aos dígitos de **0** a **9**. Mesmo

assim, a Trie aproveita o compartilhamento de prefixos entre títulos, armazenando partes comuns apenas uma vez. Assim, há uma troca entre maior uso de memória e buscas por prefixo mais eficientes.